



北京工业大学

BEIJING UNIVERSITY OF TECHNOLOGY

嵌入式系统设计技术

基础项目设计报告

学 期： 2025-2026 学年 第二学期

专 业： 计算机科学与技术（实验班）

学号姓名： 23071005 侯景祺

完成日期： 2026/4/13

教师评语					
得 分		教师签字		评阅日期	

一、测距仪

1. 功能设计

本项目设计并实现利用超声波测距技术及滤波算法的测距仪，其具有以下几项功能：

- 1) 距离测量：具备高频实时超声波距离检测功能，有效测量范围最大可达 555cm。
- 2) 数据滤波处理：内置滑动平均滤波算法，对近 5 次采样数据进行平均计算，有效减少测量数据的随机波动，提高测量精度和稳定性。
- 3) 异常报警与提示：具备测量范围超限提示（大于 555cm 报警）以及未检测到有效目标或超时等情况下的错误反馈功能。

2. 硬件电路设计

本项目以微控制器为核心控制板。通过开发板接口与超声波传感器模块连接，主控板的引脚 2 用于控制超声波传感器的 Trig 触发信号，引脚 3 用于读取超声波传感器的 Echo 回响信号。以 cm 为单位的实时测距数值和滤波处理后的数值，均以 115200 的波特率传输至上位机的串口监视器中显示。

该系统的硬件电路示意图如图 1 所示。

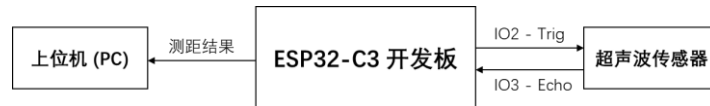


图 1 硬件电路示意图

3. 程序代码设计

```
const int TRIG_PIN = 2;
const int ECHO_PIN = 3;

const int NUM_READINGS = 5;    // 滤波采样 num
float readings[NUM_READINGS];   // 存储历史数据 数组
int readIndex = 0;              // 当前数据的索引
float total = 0;                // 历史数据总和
float averageDistance = 0;      // 滤波后的平均距离

void setup() {
    // 初始化 波特率 115200
    Serial.begin(115200);

    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);

    // 初始化滤波数组
    for (int i = 0; i < NUM_READINGS; i++) {
        readings[i] = 0; // 近五次 avg
    }
    Serial.println("初始化完成");
}
```

```

void loop() {
    // 产生 TRIG
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    // 读取 ECHO
    long duration = pulseIn(ECHO_PIN, HIGH, 70000); // 超时时间

    // 计算距离
    float distance = 0;

    if (duration == 0 || duration > 60000) {
        distance = -1.0; // 错了
    } else {
        // 算出距离
        distance = (duration * 0.034) / 2.0;
    }

    if (distance > 555) {
        Serial.println("范围超限");
    } else if (distance != -1.0) { // 滤波
        total = total - readings[readIndex];
        readings[readIndex] = distance;
        total = total + readings[readIndex];
        readIndex = readIndex + 1;

        if (readIndex >= NUM_READINGS) {
            readIndex = 0;
        }

        averageDistance = total / NUM_READINGS;

        Serial.print("实时距离: ");
        Serial.print(distance);
        Serial.print(" cm \t");
        Serial.print("滤波后距离: ");
        Serial.print(averageDistance);
        Serial.println(" cm");
    } else {
        Serial.println("传感器未检测到有效目标!");
    }
}

```

```
}

// 100ms 测一次
delay(100);
}
```

4. 测试

待测物体正面垂直于超声波传感器在不同距离下进行测试，并重点观测串口输出的“实时距离”与“滤波后距离”差异。

- 1) 静态距离测试：在有效范围内测量静态物体时，实时距离偶尔出现微小跳动，但滤波后距离 (averageDistance) 保持高度稳定。
- 2) 动态过渡测试：当待测物体距离突然大幅改变时，由于采用了 5 次滑动平均滤波，串口监视器中显示的滤波距离数据存在约 0.5 秒的平滑过渡现象，有效削弱了突发噪点的影响。
- 3) 边界及异常测试：当待测物体距离拉远超过 555cm 时，系统成功检测并输出“范围超限”。当拔掉 Echo 线或遮挡传感器导致无回波时，系统正确捕获到超时错误，并输出“传感器未检测到有效目标!”。

表 1 测距仪功能测试结果

测试条件	静态测量 (20cm)	动态距离突变	距离 > 555cm	传感器遮挡/无回波
测试结果	实时距离波动±1cm，滤波距离稳定在 20.0cm 左右	滤波后距离呈现平滑上升/下降曲线，无瞬时跳变	串口输出“范围超限”	串口输出“传感器未检测到有效目标!”

二、 电子密码箱

1. 功能设计

本项目设计并实现了一个基于触摸键盘与 OLED 显示屏的电子密码箱系统，其具有以下几项功能：

- 1)双状态安全管理：系统具备“开锁 (UNLOCKED)”与“上锁 (LOCKED)”两种状态。在开锁状态下，用户可以自由设定新密码；上锁后，必须输入正确密码方可开启。
- 2)单次有效密码逻辑：用户在开锁状态下设置的密码仅供下一次解锁使用，解锁成功后系统会自动销毁该密码，确保安全性。
- 3)管理员强制接管：内置管理员后门（密码：admin123），可通过串口指令强制重置密码箱状态，应对忘记用户密码的情况。
- 4)可视化交互界面：利用 OLED 屏幕实时显示当前状态 (OPEN/LOCKED)、输入掩码处理（上锁状态下显示“Z”字符遮蔽）及操作结果弹窗提示。
- 5)便捷操作逻辑：按键 [A] 用于确认输入，按键 [C] 用于快速清除当前输入框。

2. 硬件电路设计

本项目以微控制器为核心，采用 I2C 总线架构连接外设。系统主要包含 OLED 显示模块与 TTP229 触摸键盘模块。两个模块通过共享 I2C 总线（SDA/SCL）与主控板通信，OLED 设备的 I2C 地址为 0x3C，触摸键盘的 I2C 地址为 0x57。

该系统的硬件电路示意图如图 2 所示。

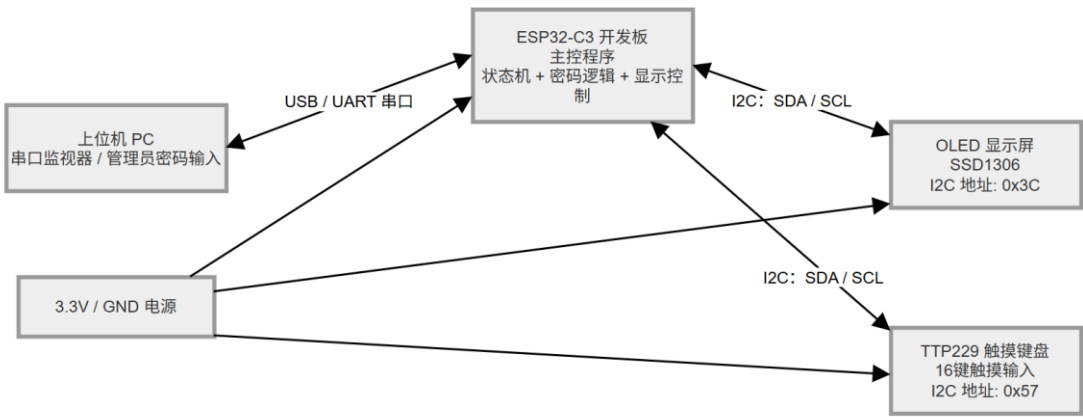


图 2 硬件电路示意图

3. 程序代码设计

本项目代码采用了有限状态机 (FSM) 的结构来处理复杂的逻辑切换，并实现了基于 I2C 的按键边缘检测。

重要变量说明：

currentState：枚举类型 `SafeState`，用于标记系统处于 UNLOCKED 或 LOCKED 状态。

savedPassword：存储用户设定的单次有效密码。

keyMap：字符数组，将 TTP229 的 16 位二进制原始数据映射为具体的数字和功能字符 (0-F)。

核心函数与逻辑说明：

readTTP229(): 通过 `Wire` 库读取键盘状态，利用 `lastKeyStatus` 实现按键边缘检测，确保一次按键仅触发一次输入，防止长按导致重复输入。

handleKeypress(char key): 核心逻辑处理器。

若处于 `UNLOCKED` 且按下 [A]，则校验输入长度 (≥ 4 位) 并将 `currentInput` 转存为新密码，切换至锁定态。

若处于 `LOCKED` 且按下 [A]，则比对输入与 `savedPassword`，匹配成功则清空密码并解锁。

updateDisplay(): 负责实时绘制 UI，根据 `currentState` 决定显示明文还是掩码，并动态刷新标题栏。

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define SCREEN_ADDRESS 0x3C
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// 触摸键盘 I2C 地址
const int TTP229_I2C_ADDRESS = 0x57;

// 键盘映射表
const char keyMap[16] = {'7', '6', '5', '4', '3', '2', '1', '0',
                        'F', 'E', 'D', 'C', 'B', 'A', '9', '8'};
uint16_t lastKeyStatus = 0; // 按键防抖 边缘检测

// 状态机枚举
enum SafeState {
    UNLOCKED,
    LOCKED
};
SafeState currentState = UNLOCKED; // 一开始归零

String currentInput = ""; // 当前输入的字符串
String savedPassword = ""; // 存入系统的一次性密码
String adminPassword = "admin123"; // 管理员密码

void setup() {
    Serial.begin(115200);
    Wire.begin(); // 初始化 I2C 总线

    // 初始化 OLED
    if(!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
```

```

    Serial.println(F("OLED 初始化失败"));
    for(;;);
}
display.setTextColor(SSD1306_WHITE);

updateDisplay(); // 刷新初始屏幕
Serial.println("电子密码箱系统启动。管理员密码为: admin123");
}

void loop() {
    // 读取触摸按键
    char key = readTTP229();
    if (key != '\0') {
        handleKeypress(key);
    }

    // 监听串口管理员指令
    handleAdminSerial();

    delay(20); // 略微延时
}

// TTP229 读取
char readTTP229() {
    Wire.requestFrom(TTP229_I2C_ADDRESS, 2);
    if (Wire.available() == 2) {
        uint8_t data0 = Wire.read();
        uint8_t data1 = Wire.read();
        uint16_t keyStatus = (data1 << 8) | data0;

        // 只有在按键刚按下时才触发, 按住不放不重复触发
        if (keyStatus != 0 && lastKeyStatus == 0) {
            lastKeyStatus = keyStatus;
            for (int i = 0; i < 16; i++) {
                if (keyStatus & (1 << i)) {
                    return keyMap[i];
                }
            }
        } else if (keyStatus == 0) {
            lastKeyStatus = 0; // 按键松开重置
        }
    }
    return '\0'; // 无有效按键
}

```

```

// 按键逻辑处理
void handleKeypress(char key) {
    if (key == 'C') {
        currentInput = "";
        updateDisplay();
    }
    else if (key == 'A') {
        if (currentState == UNLOCKED) {
            // 在开锁状态下 输入的是“设定新密码”
            if (currentInput.length() >= 4) {
                savedPassword = currentInput;
                currentState = LOCKED;        // 关门上锁
                currentInput = "";            // 清空输入框
            } else {
                showTemporaryMessage("Error!", "Password too short!", "Must be >= 4
chars");
            }
        }
        else if (currentState == LOCKED) {
            // 在上锁状态下 输入的是“解锁密码”
            if (currentInput == savedPassword) {
                currentState = UNLOCKED;    // 开锁
                savedPassword = "";          // 密码单次有效, 立即销毁
                currentInput = "";
                showTemporaryMessage("Success!", "Safe Unlocked.", "");
            } else {
                showTemporaryMessage("Denied!", "Wrong Password!", "");
                currentInput = ""; // 清空错误输入
            }
        }
        updateDisplay();
    }
    else {
        // 常规字符输入逻辑
        if (currentInput.length() < 10) { // 限制密码最大长度为 10
            currentInput += key;
            updateDisplay();
        }
    }
}

// 串口管理员控制
void handleAdminSerial() {

```



```
if (Serial.available() > 0) {
  String input = Serial.readStringUntil('\n');
  input.trim(); // 去除换行符和空格

  if (input == adminPassword) {
    // 管理员密码正确，强制重置密码箱
    currentState = UNLOCKED;
    savedPassword = "";
    currentInput = "";

    Serial.println("【管理员操作】密码箱已强制解锁，用户密码已清除。");
    showTemporaryMessage("Admin Access", "System Overridden", "Safe
Unlocked");
    updateDisplay();
  } else {
    Serial.println("【管理员操作】密码错误！拒绝访问。");
  }
}

// OLED 显示更新
void updateDisplay() {
  display.clearDisplay();

  // 绘制顶部标题栏
  display.setTextSize(1);
  display.setCursor(0, 0);
  if (currentState == UNLOCKED) {
    display.print("Status: OPEN (Set PWD)");
  } else {
    display.print("Status: LOCKED");
  }
  display.drawLine(0, 10, 128, 10, SSD1306_WHITE);

  // 绘制当前输入框
  display.setTextSize(2);
  display.setCursor(0, 25);

  if (currentInput.length() == 0) {
    display.print("_");
  } else {
    if (currentState == UNLOCKED) {
      display.print(currentInput); // 明文
    } else {
```

```

        for (int i = 0; i < currentInput.length(); i++) {
            display.print("Z");          // 掩码
        }
    }
}

// 绘制底部按键提示
display.setTextSize(1);
display.setCursor(0, 55);
display.print("[A] Confirm  [C] Clear");

display.display();
}

// OLED 临时弹窗提示
void showTemporaryMessage(String line1, String line2, String line3) {
    display.clearDisplay();
    display.setTextSize(1);
    display.setCursor(10, 15);
    display.print(line1);
    display.setCursor(10, 30);
    display.print(line2);
    display.setCursor(10, 45);
    display.print(line3);
    display.display();
    delay(1500); // 提示停留 1.5 秒
}

```

4. 测试

通过对电子密码箱的不同操作路径进行测试，验证系统可靠性：

正常流程测试：在初始开锁状态输入 1234 后按 [A]，系统正确切换为 LOCKED。随后输入 1234 按 [A]，系统弹出 Success! 弹窗并成功解锁。

安全性测试：在上锁状态下输入错误密码，系统显示 Denied! 弹窗并清空输入框，且此时屏幕上显示的输入内容均为掩码 ZZZZ。

管理员干预测试：模拟忘记用户密码，通过上位机串口发送 admin123，系统立即响应 System Overridden 并恢复至开锁状态，验证了后台强制恢复功能的有效性。

表 2 电子密码箱功能测试结果

测试条件	初始上电状态	设置新密码（长度<4）	锁定后输入正确密码	串口发送 admin123
预期结果	状态显示 OPEN	提示 Error/太短	提示 Success 并解锁	强制解锁并清空数据

测试条件	初始上电状态	设置新密码 (长度<4)	锁定后输入正确密码	串口发送 admin123
测试结果	符合预期	符合预期	符合预期	符合预期

三、 电子时钟

1. 功能设计

本实验实现了一个基于 ESP32-C3 的简易电子时钟/闹钟系统。系统上电后先初始化 I2C 总线、RTC 实时时钟模块和 OLED 显示屏，然后从 EEPROM 中读取已经保存的闹钟时、闹钟分和闹钟开关状态。正常模式下，OLED 实时显示当前时间以及闹钟设定值；用户可通过两个按键完成时间设置、闹钟时间设置以及闹钟开关控制。

程序一共设计了 6 个工作模式：0 为正常显示模式，1 为设置当前时间的小时，2 为设置当前时间的分钟，3 为设置闹钟小时，4 为设置闹钟分钟，5 为设置闹钟开关。K1 用于切换模式，K2 用于增加数值或切换开关；在设置数值时，还支持长按连续加一。退出设置模式回到正常模式时，系统会把闹钟参数重新写入 EEPROM，保证掉电后仍能保留闹钟设置。

当系统处于正常模式、闹钟开关打开、当前时分与闹钟设定相同，并且秒数小于 30 秒时，蜂鸣器开始响铃。用户按下任意按键后可以停止本次响铃，直到当前时间离开这一分钟后，系统才重新允许下一次响铃。

2. 硬件电路设计

本系统采用 ESP32-C3 作为主控芯片，外接 DS1307 时钟模块、OLED 显示屏、EEPROM、按键和蜂鸣器，完成时间显示、闹钟设置、参数存储和到时提醒等功能。各外设中，RTC、OLED 和 EEPROM 通过 I2C 总线与主控连接，K1 和 K2 分别用于模式切换和数值调整，蜂鸣器用于闹钟报警提示。

结构图如下。

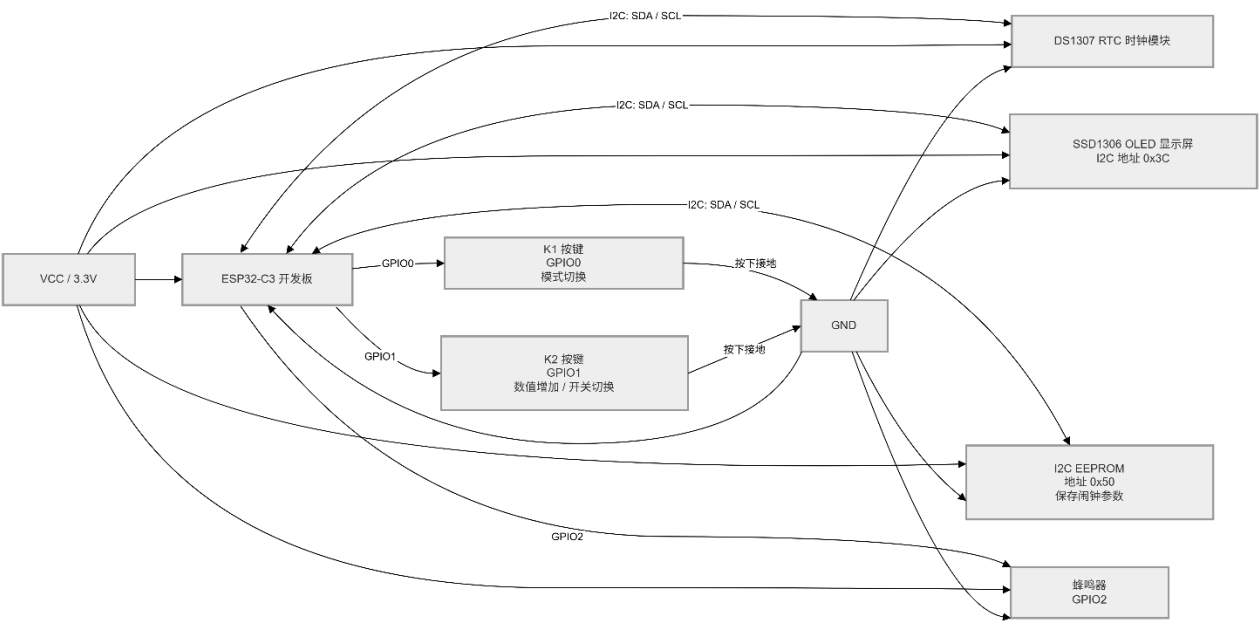


图 3 硬件电路示意图

3. 程序代码设计

(1) 初始化模块

初始化模块主要在 `setup()` 函数中完成，负责设置按键和蜂鸣器引脚模式，初始化 I2C 总线、DS1307 实时时钟模块和 SSD1306 OLED 显示屏，并在系统首次运行或 RTC 未工作时用编译时间对时。同时，程序还会从 EEPROM 中读取之前保存的闹钟小时、分钟和开关状态，并对读取到的参数做范围检查，防止出现无效数据。

(2) EEPROM 读写模块

程序通过 `writeEEPROM()` 和 `readEEPROM()` 两个函数实现对 I2C EEPROM 的访问，用来保存闹钟时间和闹钟开关状态。这样即使系统断电，下次重新启动时也能恢复之前设置好的闹钟参数，提高了系统的实用性。

(3) 显示刷新模块

显示模块由 `updateDisplay()` 函数实现。在正常模式下，OLED 显示当前时、分、秒以及闹钟时间和开关状态；在设置模式下，屏幕会显示当前所处的设置项目，例如“设置时间小时”“设置闹钟分钟”等，同时显示正在修改的时间或闹钟内容，方便用户操作。

(4) 按键控制与状态切换模块

主循环中通过检测 K1 和 K2 的电平变化来实现人机交互。K1 用于在正常模式、设置时间、设置闹钟和闹钟开关等 6 个模式之间循环切换；K2 用于在不同模式下执行加一或开关切换操作。退出设置模式时，程序会自动把新的闹钟参数写入 EEPROM 保存。

(5) 长按连发模块

在设置小时或分钟时，程序不仅支持单击加一，还支持长按连续修改。当 K2 按下超过 1 秒后，系统进入连续加一状态，并以固定时间间隔自动递增数值。这种设计能让用户更快完成时间和闹钟的设置。

(6) 闹钟判断与蜂鸣器提醒模块

程序在主循环中不断读取当前时间，并判断是否满足响铃条件：系统处于正常模式、闹钟开关打开、当前时分与设定闹钟一致、秒数小于 30 秒且本次未被手动关闭。若满足条件，则驱动蜂鸣器发声；否则关闭蜂鸣器。用户在响铃期间按下任意按键，还可以实现本次闹钟静音。

```
#include <Wire.h>
#include <RTClib.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
RTC_DS1307 rtc;

const int BTN_MODE = 0; // K1: 切换模式
const int BTN_ADD = 1;  // K2: 增加数值/切换开关
const int BUZZER = 2;   // IO2 (蜂鸣器)

// 模式扩展: 0:正常, 1:设时间时, 2:设时间分, 3:设闹钟时, 4:设闹钟分, 5:闹钟开关
int mode = 0;
int alarmHr = 0, alarmMin = 0;
bool alarmOn = false;
bool isRinging = false;
bool alarmDismissed = false;
```

```

#define EEPROM_ADDR 0x50 // 默认 7 位地址

// 有效 16bit, 2byte
void writeEEPROM(unsigned int addr, byte data) {
    Wire.beginTransmission(EEPROM_ADDR);
    Wire.write((int)(addr >> 8)); // 发高位地址
    Wire.write((int)(addr & 0xFF)); // 发低位地址
    Wire.write(data);
    Wire.endTransmission();
    delay(5);
}

byte readEEPROM(unsigned int addr) {
    byte data = 0;
    Wire.beginTransmission(EEPROM_ADDR);
    Wire.write((int)(addr >> 8));
    Wire.write((int)(addr & 0xFF));
    Wire.endTransmission();
    Wire.requestFrom((uint16_t)EEPROM_ADDR, (uint8_t)1); // 每次读 1byte
    if (Wire.available()) data = Wire.read();
    return data;
}

// 刷新屏幕显示
void updateDisplay() {
    DateTime now = rtc.now();
    display.clearDisplay();
    display.setTextColor(WHITE);
    display.setCursor(0, 0);

    if (mode == 0) {
        display.setTextSize(2);
        display.printf("%02d:%02d:%02d\n", now.hour(), now.minute(),
now.second());
        display.setTextSize(1);
        display.setCursor(0, 30);
        display.printf("Alarm: %02d:%02d [%s]", alarmHr, alarmMin, alarmOn ?
"ON" : "OFF");
    } else {
        display.setTextSize(1);
        if (mode == 1) display.println("SETTING: TIME HOUR");
        if (mode == 2) display.println("SETTING: TIME MIN");
        if (mode == 3) display.println("SETTING: ALARM HOUR");
        if (mode == 4) display.println("SETTING: ALARM MIN");
    }
}

```

```

    if (mode == 5) display.println("SETTING: ALARM ON/OFF");

    display.setTextSize(2);
    display.setCursor(0, 20);

    if (mode == 1 || mode == 2) {
        // 正在设置本地时间
        display.printf("Time: \n%02d:%02d", now.hour(), now.minute());
    } else {
        // 正在设置闹钟
        display.printf("Alrm: \n%02d:%02d [%s]", alarmHr, alarmMin, alarmOn ?
"ON" : "OFF");
    }
}
display.display();
}

void setup() {
    pinMode(BTN_MODE, INPUT_PULLUP);
    pinMode(BTN_ADD, INPUT_PULLUP);
    pinMode(BUZZER, OUTPUT);

    Wire.begin();
    rtc.begin();
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C);

    if (!rtc.isrunning()) {
        rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
    }

    alarmHr = readEEPROM(0);
    alarmMin = readEEPROM(1);
    alarmOn = (readEEPROM(2) == 1);

    if (alarmHr > 23) alarmHr = 0;
    if (alarmMin > 59) alarmMin = 0;
}

void loop() {
    DateTime now = rtc.now();

    // 如果时间已经过了闹钟设定的分钟，重置静音标志位
    if (now.hour() != alarmHr || now.minute() != alarmMin) {
        alarmDismissed = false;
    }
}

```

```

}

// K1 按键逻辑：切换模式 (0 -> 1 -> 2 -> 3 -> 4 -> 5 -> 0)
if (digitalRead(BTN_MODE) == LOW) {
    delay(50); // 基础去抖
    if (digitalRead(BTN_MODE) == LOW) {
        if (isRinging) {
            alarmDismissed = true;
        } else {
            mode = (mode + 1) % 6; // 模式拓展为 6 个
            if (mode == 0) {
                // 退出设置模式时，保存闹钟状态
                writeEEPROM(0, alarmHr);
                writeEEPROM(1, alarmMin);
                writeEEPROM(2, alarmOn ? 1 : 0);
            }
        }
        while(digitalRead(BTN_MODE) == LOW); // 等待释放
    }
}

// K2 按键逻辑：数值增加/开关切换（含长按连续修改）
if (digitalRead(BTN_ADD) == LOW) {
    delay(50); // 基础去抖
    if (digitalRead(BTN_ADD) == LOW) {

        if (isRinging) {
            alarmDismissed = true;
            while(digitalRead(BTN_ADD) == LOW);
        }
        else if (mode == 5) { // 原 mode 3 变成了 mode 5
            alarmOn = !alarmOn;
            while(digitalRead(BTN_ADD) == LOW);
        }
        else if (mode >= 1 && mode <= 4) {
            // 处于修改数值的模式：1(时间时)，2(时间分)，3(闹钟时)，4(闹钟分)

            // 1. 立即触发单次修改
            if (mode == 1) {
                rtc.adjust(DateTime(now.year(), now.month(), now.day(), (now.hour() +
1) % 24, now.minute(), now.second()));
            } else if (mode == 2) {
                // 修改分钟时，将秒数归零，方便精确对时

```



```

        rtc.adjust(DateTime(now.year(), now.month(), now.day(), now.hour(),
(now.minute() + 1) % 60, 0));
    } else if (mode == 3) {
        alarmHr = (alarmHr + 1) % 24;
    } else if (mode == 4) {
        alarmMin = (alarmMin + 1) % 60;
    }

    updateDisplay(); // 立即刷新反馈

    // 2. 记录时间, 准备检测长按
    unsigned long pressTime = millis();
    bool isLongPress = false;

    // 3. 长按连续增加逻辑
    while(digitalRead(BTN_ADD) == LOW) {
        if (!isLongPress) {
            if (millis() - pressTime > 1000) {
                isLongPress = true; // 超过1秒激活连发
            }
        } else {
            // 长按连发状态, 需要重新获取当前时间(因为 RTC 刚才可能被修改了)
            now = rtc.now();

            if (mode == 1) {
                rtc.adjust(DateTime(now.year(), now.month(), now.day(),
(now.hour() + 1) % 24, now.minute(), now.second()));
            } else if (mode == 2) {
                rtc.adjust(DateTime(now.year(), now.month(), now.day(),
now.hour(), (now.minute() + 1) % 60, 0));
            } else if (mode == 3) {
                alarmHr = (alarmHr + 1) % 24;
            } else if (mode == 4) {
                alarmMin = (alarmMin + 1) % 60;
            }

            updateDisplay();
            delay(150); // 连发速度
        }
    }
}
}
}
}

```

```

// 判断是否响铃
if (mode == 0 && alarmOn && now.hour() == alarmHr && now.minute() ==
alarmMin && now.second() < 30 && !alarmDismissed) {
    isRinging = true;
} else {
    isRinging = false;
}

// 驱动蜂鸣器
if (isRinging) tone(BUZZER, 1000);
else noTone(BUZZER);

// 正常循环中的显示更新
updateDisplay();
delay(50);
}

```

4. 测试

本实验通过对电子时钟系统的主要功能路径进行测试，验证了系统在时间显示、按键交互、闹钟设置、蜂鸣提醒以及参数保存等方面的正确性和稳定性。系统上电后，能够正常初始化 RTC、OLED 显示屏和 I2C 总线，并从 EEPROM 中读取已保存的闹钟小时、分钟以及开关状态；若 RTC 未正常运行，程序还会自动使用编译时间完成初始校时。

在基本功能测试中，OLED 可以正常显示当前时间以及闹钟状态。按下 K1 后，系统能够在正常显示模式、时间设置模式、闹钟时间设置模式和闹钟开关设置模式之间循环切换；按下 K2 后，可根据当前模式完成小时、分钟或闹钟开关的修改。其中，在设置时间分钟时，程序会自动将秒数清零，以便更准确地完成校时；在设置闹钟参数时，显示界面也能同步更新，给出直观反馈。

在交互性能测试中，系统不仅支持单次按键修改，还支持长按 K2 后连续递增数值。程序设计为按住超过 1 秒后进入连发状态，并以固定时间间隔持续增加当前设置值，因此在实际使用中能够明显提高调时和设定闹钟的效率。测试结果表明，长按功能工作正常，未出现数值跳变异常或界面卡死现象。

在闹钟功能测试中，当系统处于正常显示模式、闹钟开关打开、当前时分与设定闹钟一致且秒数小于 30 秒时，蜂鸣器能够正常发声；当用户在响铃期间按下任意按键后，系统可立即停止本次响铃，并在离开该闹钟分钟后重新恢复下一次提醒能力。这说明程序中的闹钟触发逻辑与静音逻辑均能正确工作。

此外，在掉电保存测试中，退出设置模式回到正常模式后，系统会将闹钟小时、分钟及开关状态写入 EEPROM。重新上电后，系统能够重新读出这些参数并恢复先前设置，说明本实验实现了较好的非易失化存储功能。综合测试结果可知，该电子时钟系统能够较稳定地完成时间显示、闹钟设置、响铃提醒和参数保存等预期功能。

表 3 电子时钟功能测试结果

测试条件	初始上电状态	K1 切换模式	设置当前时间	设置闹钟时间	设置闹钟开关
预期结果	显示当前时间和闹钟状态	模式依次切换	小时/分钟可修改	闹钟时/分可修改	ON/OFF 可切换
测试结果	符合预期	符合预期	符合预期	符合预期	符合预期

表 4 电子时钟报警功能测试结果

测试条件	闹钟到时触发	响铃时按键消音	长按 K2 连续加一	掉电后重新上电
预期结果	蜂鸣器在当前时分与闹钟一致且秒数 小于 30 时响铃	按下任意按键后停止 本次响铃	按住超过 1 秒后可连续 递增数值	系统恢复先前保存的 闹钟参数
测试结果	符合预期	符合预期	符合预期	符合预期

四、 公交车报站器

1. 功能设计

本实验设计并实现了一个基于 ESP32-C3 的公交车报站器系统。系统通过按键触发站点切换，每按下一次按键，系统就在 3 个预设站名之间循环切换，分别为“北太平庄”“永定门外”和“西直门”。在切换站点时，ESP32-C3 一方面通过串口向语音播报模块发送对应站名的数据帧，实现语音播报；另一方面通过 LED 点阵依次显示当前站名中的各个汉字，从而实现“语音 + 文字”的同步报站。

本系统采用单按键控制方式，用户操作简单。程序中使用定时器中断配合按键边沿检测，只在按键由高电平变为低电平的瞬间触发一次报站，因此能够避免长按时重复误触发的问题。为了在 LED 点阵上显示汉字，系统先根据站名字的 GB2312 编码计算其在字库芯片中的物理地址，再通过 SPI 读取对应字模数据，并分配到左右两个 8×16 LED 点阵上显示。整体上，本实验完成了公交车到站播报、站名文字显示和按键切换站点等基本功能。

2. 硬件电路设计

本实验硬件系统以 ESP32-C3 开发板为核心，外接按键输入模块、语音播报模块、汉字字库芯片和两块 8×16 LED 点阵模块。按键 K1 连接到 ESP32-C3 的 GPIO0，用于触发站点切换；左右两块 LED 点阵分别通过 I2C 总线连接，地址设置为 0x70 和 0x71；汉字字库芯片通过 SPI 总线与主控通信，其中片选信号接 GPIO7；语音播报模块通过 UART 串口与 ESP32-C3 相连，用于输出对应站名的语音内容。

系统工作时，ESP32-C3 负责接收按键信号、维护当前站点序号、向语音模块发送播报数据帧，并控制字库读取与 LED 点阵显示。各模块分工明确，接口关系清晰，适合实现一个简易的公交站名播报装置。

硬件电路图如下。

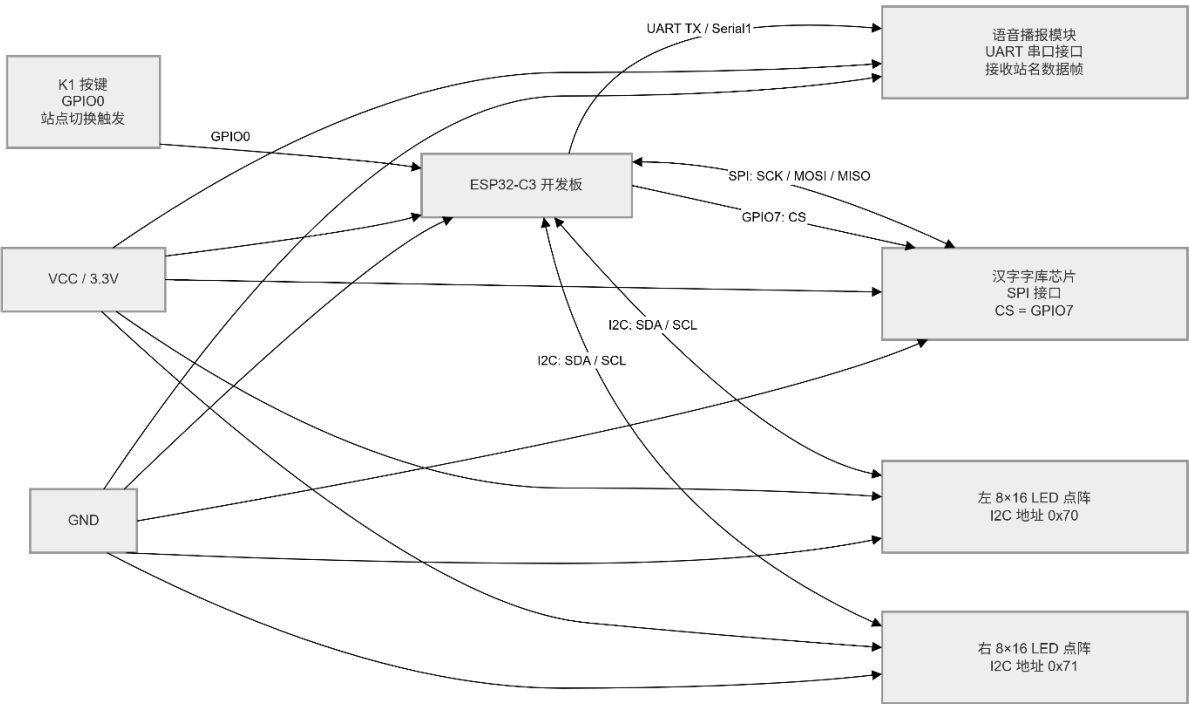


图 4 硬件电路示意图

3. 程序代码设计

本实验程序主要由站名编码定义、字模地址换算、按键中断处理、字库读取、点阵显示和主循环报站控制等几个部分组成。首先，程序为“北太平庄”“永定门外”“西直门”等站名中的每个汉字定义了对应的 GB2312 编码，同时构造了发送给语音播报模块的串口数据帧。其次，程序通过 `findByteAddr()` 函数将汉字区位码换算成字库芯片中的实际存储地址，再利用 `led()` 函数通过 SPI 读取 32 字节字模数据，并拆分到左右两个 LED 点阵显示缓冲区中。

在控制逻辑方面，程序使用硬件定时器周期性检测按键状态，并在中断服务函数中完成按键边沿检测。当检测到 K1 被按下时，系统更新当前站点编号，并在主循环中根据编号执行对应站点的语音播报与文字显示。显示时，系统会依次取出站名中的各个汉字，在 LED 点阵上逐字显示，完成后再清屏，从而形成简单直观的报站效果。

```
#include <Wire.h>
#include <SPI.h>
#include <Adafruit_LEDBackpack.h>

#define K1 0
#define LEFT 0x70
#define RIGHT 0x71
#define HZK_CS 7
#define BASE_ADDR 0x2C9D0 // P15

// 站名的 GB2312 十六进制编码
#define BEI 0xB1B1 // 北
#define TAI 0xCCAB // 太
#define PING 0xC6BD // 平
#define ZHUANG 0xD7AF // 庄

#define YONG 0xD3C0 // 永
#define DING 0xB6A8 // 定
#define MEN 0xC3C5 // 门
#define WAI 0xCDE2 // 外

#define XI 0xCEF7 // 西
#define ZHI 0xD6B1 // 直

hw_timer_t * timer = NULL;

// 跨中断共享变量必须加 volatile
volatile bool bo;
volatile int lop;
volatile bool last_btn_state = HIGH; // 按键边缘检测
```

```

Adafruit_8x16matrix matrixl = Adafruit_8x16matrix();
Adafruit_8x16matrix matrixr = Adafruit_8x16matrix();
uint8_t BUF[128];
byte led_arr[32];
byte ledl[16], ledr[16], zeros[16] = {0}; // 显式初始化全 0
int word_addr;

// 拆分高低字节
int msb(int addr){
    return (addr & 0xFF00) >> 8;
}

int lsb(int addr){
    return addr & 0x00FF;
}

int findByteAddr(int addr){ // GB2312 区位码转物理地址
// 乘 32 是一个汉字刚好 256 个点, 偏移量
    int MSB = msb(addr);
    int LSB = lsb(addr);
    // 高低字节推算在矩阵中的位置, 乘空间得到内存 addr
    if(MSB >= 0xA1 && MSB <= 0xA9 && LSB >= 0xA1)
        return ((MSB - 0xA1) * 94 + (LSB - 0xA1)) * 32 + BASE_ADDR;
    else if (MSB >= 0xB0 && MSB <= 0xF7 && LSB >= 0xA1)
        return ((MSB - 0xB0) * 94 + (LSB - 0xA1) + 846) * 32 + BASE_ADDR;

    // 防止查表失败时返回不可控的内存垃圾值
    return BASE_ADDR;
}

// 格式 帧头 数据长度*2 命令字 文本编码格式
// 4 个汉字(8 字节) -> 长度为 0x0A (10), 数组总大小 13
uint8_t BTPZ[13] = {0xFD, 0x00, 0x0A, 0x01, 0x00, msb(BEI), lsb(BEI),
msb(TAI), lsb(TAI), msb(PING), lsb(PING), msb(ZHUANG), lsb(ZHUANG)};
// 4 个汉字(8 字节) -> 长度为 0x0A (10), 数组总大小 13
uint8_t YDMW[13] = {0xFD, 0x00, 0x0A, 0x01, 0x00, msb(YONG), lsb(YONG),
msb(DING), lsb(DING), msb(MEN), lsb(MEN), msb(WAI), lsb(WAI)};
// 3 个汉字(6 字节) -> 长度为 0x08 (8), 数组总大小 11
uint8_t XZM[11] = {0xFD, 0x00, 0x08, 0x01, 0x00, msb(XI), lsb(XI),
msb(ZHI), lsb(ZHI), msb(MEN), lsb(MEN)};

```

```

// ESP32 中断服务函数必须放在 IRAM 中
void IRAM_ATTR onTimer() {
    bool current_state = digitalRead(K1);
    // 边缘检测：只有在按键从 HIGH 变到 LOW 的那一瞬间才触发
    if(current_state == LOW && last_btn_state == HIGH) {
        bo = true;
        lop = (lop + 1) % 3;
    }
    last_btn_state = current_state;
}

void setup() {
    Serial.begin(115200);

    Serial1.begin(115200);

    SPI.begin();

    // 汉字库
    pinMode(HZK_CS, OUTPUT);
    digitalWrite(HZK_CS, HIGH);

    // LED 点阵初始化
    matrixl.begin(LEFT);
    matrixr.begin(RIGHT);

    // 初始设为 2，第一下按键触发时 (2+1)%3 刚好等于 0，从而正常播报第 0 个站
    lop = 2;
    bo = false; // 初始不上电就触发，等待按键

    pinMode(K1, INPUT_PULLUP);

    timer = timerBegin(1000000);

    // 绑定中断服务函数：只传 2 个参数
    timerAttachInterrupt(timer, &onTimer);

    // 设置报警并启动：参数为 (定时器指针, 报警阈值, 是否自动重载, 重载次数)
    // 50000 微秒 = 50 毫秒 (如果是 200000 则是 200 毫秒)。最后的 0 代表无限次重载触
    发。
    timerAlarm(timer, 50000, true, 0);
}

void led(int addr) {

```

```

int l = 0, r = 0;
digitalWrite(HZK_CS, LOW);
SPI.transfer(0x03); // 普通读, 不用额外的空字节发送
// 分3次 每次8位发送地址
SPI.transfer(addr >> 16);
SPI.transfer(addr >> 8);
SPI.transfer(addr & 0xFF);
for(int i = 0; i < 32; i++)
    led_arr[i] = SPI.transfer(0x00); // read
digitalWrite(HZK_CS, HIGH);

// 按行存取
for(int i = 0; i < 32; i++){
    if(i % 2) ledr[r++] = led_arr[i]; // 奇数存右屏
    else ledl[l++] = led_arr[i]; // 偶数存左屏
}
}

void disp(bool clean){
    matrixl.clear();
    matrixr.clear();
    if(clean){ // 写入空
        matrixl.drawBitmap(0, 0, zeros, 8, 16, LED_ON);
        matrixr.drawBitmap(0, 0, zeros, 8, 16, LED_ON);
    }
    else{
        matrixl.drawBitmap(0, 0, ledl, 8, 16, LED_ON);
        matrixr.drawBitmap(0, 0, ledr, 8, 16, LED_ON);
    }
    matrixl.writeDisplay();
    matrixr.writeDisplay();
    delay(300);
}

void loop() {
    if(bo){
        bo = false; // 进来后立即重置标志位, 防止中断再次触发导致状态混乱
        int current_lop = lop; // 缓存当前模式, 防止在长时间 delay 中途被中断修改

        if(current_lop == 0){ // 1. 北太平庄
            Serial1.write(BTPZ, 13);
            led(findByteAddr(BEI)); disp(false);
            led(findByteAddr(TAI)); disp(false);
            led(findByteAddr(PING)); disp(false);

```



```
        led(findByteAddr(ZHUANG)); disp(false);
        disp(true);
    }
    else if(current_lop == 1){ // 2. 永定门外
        Serial1.write(YDMW, 13);
        led(findByteAddr(YONG)); disp(false);
        led(findByteAddr(DING)); disp(false);
        led(findByteAddr(MEN)); disp(false);
        led(findByteAddr(WAI)); disp(false);
        disp(true);
    }
    else if(current_lop == 2){ // 3. 西直门
        Serial1.write(XZM, 11);
        led(findByteAddr(XI)); disp(false);
        led(findByteAddr(ZHI)); disp(false);
        led(findByteAddr(MEN)); disp(false);
        disp(true);
    }
}
}
```

4. 测试

本实验通过对公交车报站器的不同操作路径进行测试，验证系统在按键触发、站点切换、语音播报和点阵显示等方面的可靠性。测试结果表明，系统上电后能够保持等待触发状态；按下 K1 后，系统可以按照预设顺序在 3 个站点之间循环切换，并正确发送对应的串口报站数据。同时，左右 LED 点阵能够依次显示当前站名中的各个汉字，显示结束后自动清屏。

此外，为了防止按键长按导致重复触发，程序中引入了按键边沿检测机制。测试表明，即使用户持续按住 K1，不会在一次长按过程中产生多次连续播报，而只有在按键释放后再次按下时，才会切换到下一个站点。这说明程序在按键处理和状态切换方面工作正常，能够满足基本的公交报站功能需求。

表 5 公交车报站器基本功能测试结果

测试条件	初始上电状态	第一次按下 K1	第二次按下 K1	第三次按下 K1	第四次按下 K1
预期结果	系统等待触发，不自动播报	播报“北太平庄”，点阵依次显示“北”“太”“平”“庄”	播报“永定门外”，点阵依次显示“永”“定”“门”“外”	播报“西直门”，点阵依次显示“西”“直”“门”	重新回到“北太平庄”，进入循环
测试结果	符合预期	符合预期	符合预期	符合预期	符合预期

表 6 公交车报站器可靠性测试结果

测试条件	长按 K1 不放	站名显示结束后	连续多次按键切换	串口播报与点阵显示同步
预期结果	仅触发一次，不重复连续报站	LED 点阵自动清屏	站点按照固定顺序循环切换	语音播报与当前显示站名一致
测试结果	符合预期	符合预期	符合预期	符合预期

五、 小结

本次课程设计共完成了多个综合实验项目。在实验过程中，除了实现基本功能外，更重要的是学会了从“现象—定位—分析—解决”的角度处理实际问题。整体来看，实验中遇到的问题主要集中在硬件接口连接、模块通信方式理解、显示器件驱动以及程序逻辑控制几个方面。例如，在按键控制类实验中，需要考虑按键抖动、状态切换和重复触发问题；在时钟与存储类实验中，需要同时处理 I2C 设备初始化、参数保存和显示刷新等多个环节。这些问题说明，嵌入式系统设计不仅要关注功能是否实现，还要关注模块之间是否能够稳定协同工作。

其中，“公交车报站器”实验给我的印象最深，也最能体现调试过程的完整性。该实验需要同时完成按键触发、串口报站、字库读取和 LED 点阵显示等功能，看起来逻辑比较清晰，但在实际调试中，显示部分出现了明显问题。由于我一开始对 LED 点阵屏的显示方向、行列扫描方式以及模块内部驱动芯片类型不够熟悉，对屏幕的实际映射关系判断也不够准确，导致程序运行后出现了显示内容方向错误、局部区域翻转，甚至屏幕被分成 4 个区域后显示反转等现象。这个问题一开始并不是简单的“接线错误”，而是硬件结构理解不到位和显示数据排列方式不匹配共同造成的。

在发现问题后，我先通过观察显示结果，判断错误不是出现在站名字模读取本身，而更可能出现在字模数据拆分、左右屏分配以及点阵方向映射这些环节。随后，我通过查阅网络资料和相关模块的驱动说明，进一步了解了 LED 点阵屏的显存组织方式、字节排列顺序以及不同芯片的显示方向差异。在此基础上，我重新检查了显示函数中数据写入左右屏缓冲区的方式，并对屏幕显示代码进行了针对性修改，最终解决了汉字显示方向混乱和区域反转的问题，使报站器能够正确完成站名显示与语音播报。

通过这次实验，我体会到嵌入式开发中“功能代码能运行”并不等于“系统就已经调通”，很多问题往往出现在对底层硬件细节理解不够深入的地方。遇到问题时，不能只依赖反复试错，更需要结合现象分析问题来源，查阅资料验证判断，再有针对性地修改程序。整体来说，这几次实验不仅提高了我对 I2C、SPI、UART 等接口的理解，也让我对按键逻辑、状态机设计、显示驱动和系统联调过程有了更直观的认识，为后续进行更复杂的嵌入式系统开发打下了基础。